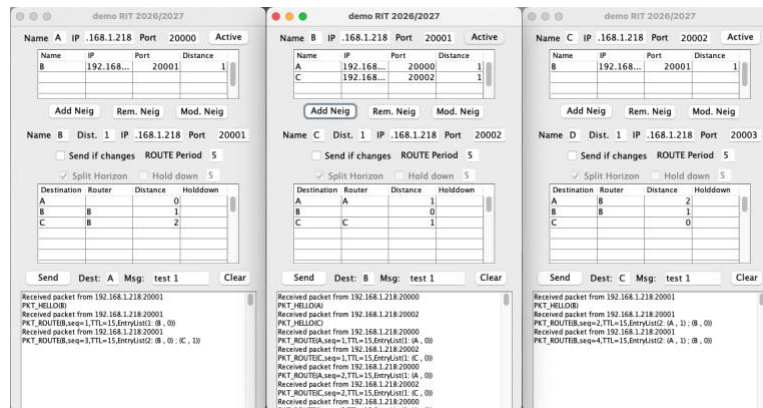




Redes Integradas de Telecomunicações 2026/2027

Trabalho 1: Encaminhamento dinâmico numa rede em malha

Aulas 2 a 5

*Mestrado em Engenharia Eletrotécnica e de Computadores
/MERSIM*

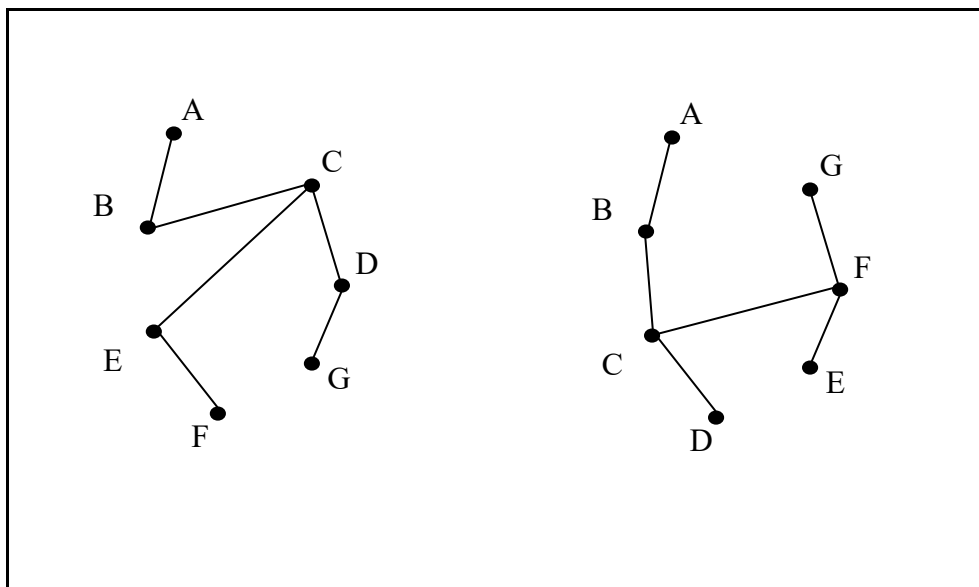
1. Objetivos

Familiarização com o funcionamento de encaminhadores (*routers*) numa rede em malha. O trabalho consiste na comunicação entre componentes interligados numa rede em malha. Neste trabalho imagina-se que as estações estão a usar uma rede em malha por cima da rede física Ethernet. O trabalho consiste no desenvolvimento de um programa ***router*** que pode comunicar com os *routers* vizinhos, oferecendo um serviço de encaminhamento baseado no **algoritmo de vetores de distâncias**, resultante de uma simplificação do algoritmo IGRP (*Inter-Gateway Routing Protocol*), proposto pela Cisco, usado nas redes TCP/IP.

Em sistemas reais, os encaminhadores interligam máquinas pertencentes a várias redes locais. Neste trabalho vai-se simular este ambiente, com algumas simplificações: os *routers* são também, simultaneamente, os emissores e recetores das mensagens. Ao gerar pacotes a partir dos vários *routers* para todos os outros *routers*, o trabalho vai permitir testar o comportamento do serviço de encaminhamento perante modificações na rede.

2. Especificações

Cada processo *router* na rede é identificado por um nome: A, B, C, D, ..., Z. Embora se esteja a usar uma rede Ethernet no laboratório ou um computador em casa, de um ponto de vista lógico os *routers* não podem comunicar diretamente uns com os outros, mas têm um circuito para o fazer. A topologia de cada rede em malha é definida pelo conjunto de relações de vizinhança introduzidas localmente na interface gráfica de cada *router*. Cada *router* tem apenas conhecimento dos vizinhos diretos, recebendo informações acerca do resto da rede apenas através do algoritmo de encaminhamento usado – vetores de distâncias. As figuras em baixo mostram duas hipóteses de redes usando 7 *routers*.



Os vários processos ***router*** de cada máquina comunicam através de *sockets* datagrama.

Um *router* real monitoriza os vizinhos na rede e, periodicamente ou em resposta a acontecimentos imprevistos, troca com eles pacotes de controlo (com as respetivas tabelas de encaminhamento). Nesta simulação, os processos *routers* simulam este comportamento através de comandos na interface gráfica de utilizador.

Um *router* oferece para o utilizador opções de monitorização dos vizinhos:

- permite acrescentar vizinhos (identificados pelo endereço IP e porto do seu *socket*);
- permite mudar a distância a qualquer dos vizinhos;
- permite desligar uma ligação para um vizinho.

O processo *router* envia periodicamente os pacotes com a informação local de encaminhamento para os vizinhos e recalcula a sua tabela local usando o algoritmo de vetores de distâncias. Para facilitar a visualização do estado do *router*, o conteúdo da tabela de encaminhamento local é apresentado na interface gráfica.

Finalmente, o processo *router* oferece na sua interface opções de envio e receção de pacotes de dados:

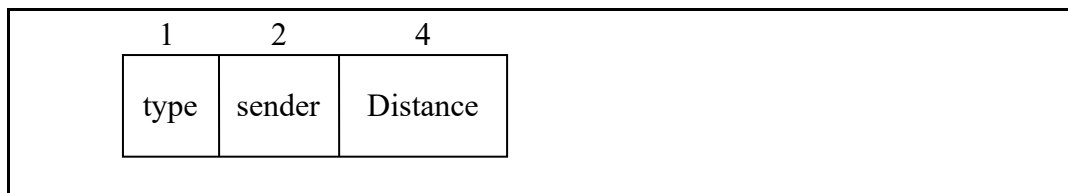
- permite enviar pacotes de dados para qualquer destino;
- recebe e reenvia os pacotes de dados para um *router* vizinho, consoante o conteúdo da tabela de encaminhamento, memorizando no pacote o percurso;
- recebe os pacotes de dados no *router* de destino, apresentando a rota completa percorrida pelo pacote desde a origem até ao destino.

Notar que neste simulador, tal como num sistema real, a topologia da rede pode ser diferente para cada sentido. Por exemplo, numa linha de acesso a uma rede com um servidor web muito acedido, o tráfego será mais elevado no sentido originado a partir do servidor web.

2.1. Configuração da rede

Quando o programa *router* arranca, sabe o seu endereço e não tem vizinhos. Depois, vai ganhar ou perder vizinhos, à medida que o utilizador usa as operações de acrescentar, remover ou modificar distâncias, ou à medida que outros *routers* se associam a ele. É usado um protocolo para criar e destruir relações de vizinhança entre *routers*. Quando um utilizador acrescenta um vizinho à lista, o programa envia um pacote HELLO para o endereço IP e porto do vizinho. Quando um programa *router* recebe um pacote HELLO acrescenta o emissor do pacote à lista de vizinhos, memoriza a lista de áreas do vizinho, respondendo também com um pacote HELLO, com igual valor de distância e com as áreas a que o nó pertence. Admite-se que por omissão a distância é igual nos dois sentidos, embora o utilizador possa modificar em qualquer altura a distância em cada sentido. Caso o vizinho não aceite o pacote HELLO por exceder o número máximo de vizinhos, termina a relação de vizinhança enviando um pacote BYE (ver adiante).

```
Pacote HELLO: { sequência contígua de }
byte          type;           { tipo pacote - HELLO = 10 }
char          sender;         { nome router origem }
int           distance;       { distância ao vizinho }
```



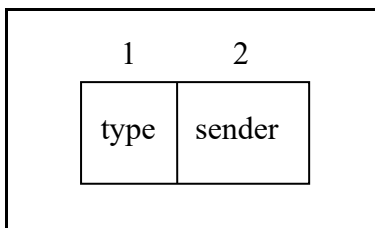
O pacote HELLO também é usado para comunicar a um vizinho que mudou a distância entre dois *routers*. A distância é um valor inteiro compreendido entre 0 e 20. **A distância 20 é reservada e simboliza a não existência de ligação entre dois routers.** Não devem ser usadas distâncias superiores a 20 ou inferiores a 0.

Quando um *router* quer terminar uma relação de vizinhança envia um pacote BYE para o vizinho. Tanto o emissor como o recetor da mensagem devem terminar imediatamente o envio de pacotes com o estado de linha (ROUTE) para o ex-vizinho.

```

Pacote BYE: { sequência contígua de }
byte      type;          { tipo pacote - BYE = 20 }
char      sender;        { nome router origem }

```



Durante o trabalho, pretende-se testar o comportamento dos algoritmos de vetores de distâncias com o surgir de novos *routers* e com o terminar de ligações ou *routers*. Assim, durante o desenvolvimento do trabalho, deverá testar o comportamento do programa desenvolvido perante estes cenários, uma vez que isso será avaliado na discussão final. Observe-se que os *routers* também podem falhar de forma silenciosa (sem enviarem BYE), e o sistema deve saber detetar este tipo de falha.

2.2. Protocolo de encaminhamento

A tarefa mais importante do programa *router* é o cálculo da tabela de encaminhamento utilizando o algoritmo de vetores de distâncias. A partir do momento em que fica ativo, o programa *router* deve enviar periodicamente para todos os vizinhos um pacote ROUTE, com o conteúdo da sua tabela local de encaminhamento. Antes de enviar um pacote ROUTE, o *router* deve recalculer a tabela de encaminhamento, utilizando a informação dos pacotes ROUTE recebidos dos vizinhos.

Os *routers* guardam sempre o pacote ROUTE mais recentes de cada router vizinho, até um tempo máximo igual ao período de vida do pacote (TTL). O valor de TTL é **igual à duração do período de envio de pacotes ROUTE mais dez segundos**.

Pacote ROUTE: { sequência contígua de }

```

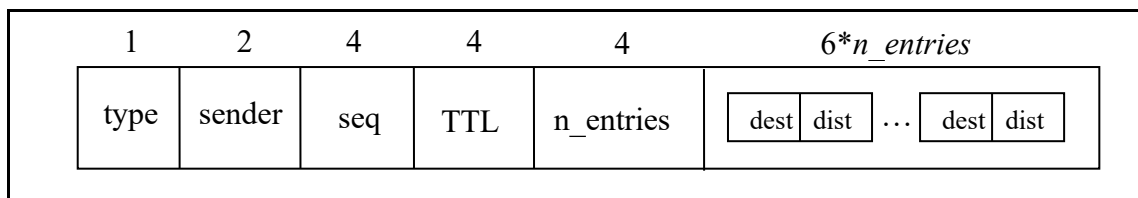
byte      type;          { tipo pacote - ROUTE = 30 }
char      sender;        { nome router origem }
int       seq;           { número de sequência }
int       TTL;           { tempo de vida da informação [s] }
int       n_entries;     { número de entradas na tabela }
Entry[]   entries;       { array com n_entries entradas }

```

```

class Entry { // Tipo de entrada no array
char      dest;          { nome de router vizinho }
int       dist;          { distância até router - entre 0 e 20 }
}

```



Para simplificar o programa, deverá admitir que as únicas modificações à topologia da rede (aos vizinhos) ocorrem como resultado de modificações controladas pelo utilizador na GUI ou da receção de pacotes HELLO e BYE. No caso da modificação ocorrer num nó não vizinho, só se deteta através dos pacotes ROUTE recebidos. Admita ainda que não há perda de pacotes na rede local onde as aplicações estão a correr.

2.3. Lidando com o problema de convergência lenta

O algoritmo de encaminhamento por vetor de distâncias converge lentamente após a falha de um troço ou de um encaminhador –que pode dar origem ao problema da contagem para o infinito. Para lidar com este problema, foram propostas várias soluções parciais para acelerar a convergência.

Uma das soluções mais difundidas é denominada "Horizontes Partidos" (*Split Horizon*). Se um encaminhador A usa uma rota até E que passa por B, então A anuncia ao encaminhador vizinho B que não conhece nenhuma rota até E **omitindo o destino no vetor enviado**. Desta forma, garante-se que B nunca vai escolher A para chegar até E, resolvendo o problema numa rede linear, sem ciclos. No entanto, se existirem circuitos fechados na rede, então o algoritmo vai continuar a apresentar problemas de convergência lenta.

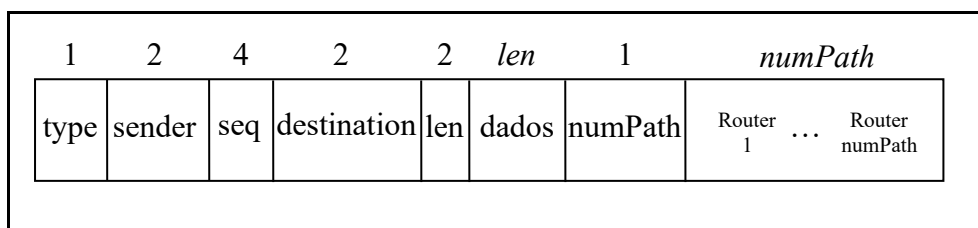
Outra solução é designada de "Aguentar" (*Hold down*). Neste caso, após ser detetada uma falha numa rota para um destino, o encaminhador vai aguentar o valor de infinito na tabela durante um intervalo de tempo fixo (difundindo o valor de infinito para os vizinhos) antes de começar a procurar por uma nova rota para esse destino, dando tempo para que o valor infinito se propague na rede.

Neste trabalho prático vão ser realizados ambos os algoritmos e avaliar até que ponto são efetivos a lidar com o problema da contagem para o infinito.

2.4. Envio e receção de dados

Os programas *router* também simulam o papel de emissor e recetor de pacotes de dados. Sempre que o utilizador seleciona a opção de enviar dados, o programa cria um pacote do tipo DATA, com o valor de *path* igual ao nome local. Em seguida, o programa consulta a tabela de encaminhamento e envia o pacote para o próximo *router* indicado na tabela, ou retorna um código de erro, caso o nome de destino esteja inacessível.

```
Pacote DATA: { sequência contígua de }
byte          type;          { tipo pacote - DATA = 40 }
char          sender;        { nome router origem }
int           seq;           { número de sequência }
char          destination;    { nome router destino }
short         len;           { número de bytes de dados }
byte[]        dados[];       { len bytes de dados }
byte          numPath;        { número de routers percorridos }
char[]        path;          { sequência de routers percorridos }
```



Cada *router* que recebe o pacote deve acrescentar o seu nome ao campo *path* do pacote, incrementando o valor de *numPath*. Caso seja o *router* de destino de pacote, deve escrever o conteúdo do pacote recebido. Senão, deve consultar a sua tabela de encaminhamento e enviar para o próximo *router*. Caso o tamanho do percurso percorrido pelo pacote atinja a distância máxima (10), então deverá aparecer uma mensagem de erro a sinalizar o facto ao utilizador, e o pacote deve ser tratado como se tivesse atingido o destino. Desta forma, fica-se sempre a saber por onde ele viajou.

3. Desenvolvimento do programa

O trabalho deve ser desenvolvido utilizando o ambiente de desenvolvimento Apache Netbeans IDE e com o JDK 21. Podem ser usados outros IDE, mas perde-se a possibilidade de edição da GUI através do editor do Netbeans.

Por outro lado, deve ser obrigatoriamente usada a ferramenta **git** durante o desenvolvimento do projeto para guardar versões intermédias do projeto, possibilitando a recuperação de código apagado por erro posteriormente.

Sugere-se que os estudantes criem uma conta no github.com¹ e um projeto privado para guardar o trabalho, de forma a salvaguardar contra falhas na máquina utilizada no desenvolvimento do projeto.

3.1. Instalação da aplicação

A aplicação *router* deve ser instalada a partir do [github.com](https://github.com/lflbernardofct/router_rit.git) clonando o projeto que está em: https://github.com/lflbernardofct/router_rit.git.

O Apache Netbeans suporta esta funcionalidade ao selecionar a opção “Team” > “Git” > “Clone ...” indicando o URL do repositório e selecionando a diretoria do computador local onde o código vai ser colocado².

3.2. Gestão de versões do software

Um assunto muito importante quando se desenvolve software é a gestão de versões. A evolução de versões ao longo do desenvolvimento pode ser linear, ou pode ser mais complexa permitindo ter caminhos divergentes e unirem-se algures no futuro. Existem alguns sistemas poderosos de gestão de versões. Um deles é o **git**, que está integrado com o github.com. O Netbeans permite usar nativamente o **git** para guardar várias versões intermédias do trabalho.

Ao instalar o projeto a partir do github.com, a configuração do **git** fica logo feita. Caso não consiga fazer a configuração dessa forma, é necessário inicializar a ferramenta **git** para o projeto instalado a partir do código fonte. A configuração está descrita no tutorial indicado em baixo, mas é muito simples: A configuração **git** do projeto é criada ao clicar o botão direito do rato depois de selecionar o projeto (neste caso o projeto *router*). Deve escolher as opções “Versioning” > “Initialize Git Repository ...”. De seguida, deve guardar uma primeira versão do trabalho. Desta forma, o projeto existente é definido como a versão-base do projeto.

Durante o desenvolvimento do trabalho, deve fazer *commit* (selecionando a opção “Git” > “Commit ...” no projeto), para guardar as versões intermédias do projeto relevantes. No enunciado são indicados alguns commits que devem ser obrigatoriamente realizados e os nomes que devem ser associados. Numa utilização mais avançada também poderá criar novos ramos do projeto (“*branches*”), que mais tarde podem ser reunidos ao projeto inicial.

Usando a opção “**git**” do projeto (com o botão direito do rato), pode ver o que foi alterado entre versões ou reverter as modificações. É muito útil quando alguma coisa deixa de funcionar.

Recomenda-se que consulte o tutorial seguinte, para aprender a usar as funcionalidades de controlo de versão do **git** no Netbeans (todos os mecanismos são muito intuitivos):

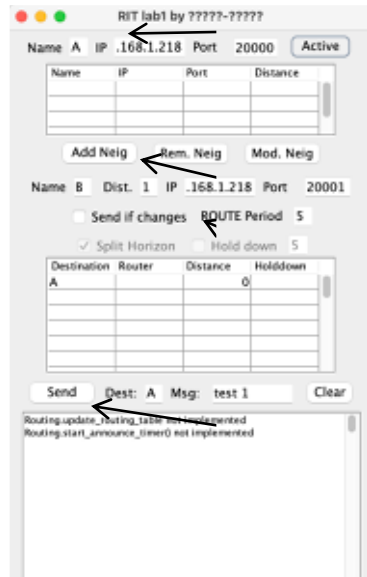
<https://netbeans.apache.org/tutorial/main/kb/docs/ide/git/>

3.3. Código fornecido

¹ As instruções para criar uma conta estão em: <https://docs.github.com/en/account-and-profile/how-tos/account-management/creating-an-account-on-github>

² Também é possível clonar através da linha de comando se instalar a aplicação git (no Linux, MacOS ou Windows) e usar o comando “*git clone https://github.com/lflbernardofct/router_rit.git*”.

Para facilitar o desenvolvimento do programa e tornar possível a sua realização durante as oito horas previstas, é fornecido juntamente com o enunciado um programa *router* incompleto, com a interface gráfica representada abaixo, que já realiza parte das funcionalidades pedidas. Cada grupo pode fazer todas as modificações que quiser ao programa base, ou mesmo desenhar uma interface gráfica de raiz. No entanto, recomenda-se que invistam o tempo na correta realização do algoritmo de encaminhamento.



Nome, IP e porto do *router*

Tabela de vizinhos

Arranque da aplicação

Dados sobre vizinho

Controlo dos vizinhos

Período envio rota

Controlo algoritmo DV

Tabela encaminhamento

Envio de pacotes de dados

Área de mensagens

O programa fornecido é composto por dez classes:

- *Log.java* (completa) – Interface para escrita de mensagens no écran;
- *Entry.java* (completa) – Elemento do vetor de vizinhos num pacote ROUTE;
- *Neighbour.java* (completa) – Descritor de vizinho, usado no controlo de vizinhança, manutenção dos vetores de distâncias e no envio de pacotes;
- *NeighbourList.java* (completa) – Descritor de lista de vizinhos, usado no controlo de vizinhança e no envio de pacotes;
- *RoutingTable.java* (completa) – Guarda uma tabela de encaminhamento. Oferece funções para manipular o conteúdo da tabela;
- *LocalIPAddresses.java* (completa) – Gere endereços IP locais e valida os endereços recebidos;
- *UnicastDaemon.java* (completa) – *thread* que recebe pacotes unicast;
- *RouteEntry.java* (completa) – Descritor de elementos da tabela de encaminhamento, que realiza o controlo do tempo de *hold down*;
- *Routing.java* (a completar) – Classe responsável pelo envio e receção de pacotes ROUTE, pelo cálculo da tabela de encaminhamento, e pelo tratamento dos pacotes DATA;
- *Router.java* (completa) – Classe principal com interface gráfica (herda de *JFrame*), que faz a gestão de sincronismo dos vários objetos usados; também gere uma *thread* para a receção de dados unicast.

O programa fornecido inclui todos os ficheiros completos exceto o ficheiro *Routing.java* que deve ser completado pelos estudantes. A ativação dos sockets e da *thread* de receção de dados já é feita no programa fornecido. Falta apenas realizar os aspetos relacionados com o algoritmo de encaminhamento (controlo do envio e processamento de pacotes ROUTE, cálculo da tabela de encaminhamento). **Uma das principais dificuldades do trabalho é precisamente ler e compreender todo o código fornecido para poder usá-lo convenientemente desde a primeira aula de laboratório.**

A classe *Routing* define um conjunto inicial de campos com os dados obtidos a partir da interface gráfica, inicializados no construtor da classe:

```
private char local_name;           // Local name
private NeighbourList neig;        // Neighbour list
private int period;                // Routing update period (ms)
private boolean splitHorizon;      // If it uses Split Horizon
private boolean holdown;           // If it uses hold down
private int holdown_time;          // Duration of hold down time (s)
private Router win;                // Main window
private DatagramSocket ds;         // Unicast datagram socket
private JTable tableObj;           // window with routing table
```

A classe *RoutingTable* encapsula a tabela de encaminhamento. Internamente define a variável *rtab* onde que é guardada a tabela de encaminhamento:

```
private final ConcurrentHashMap<Character, RouteEntry> rtab;
```

A classe *RoutingTable* define os seguintes métodos para lidar com tabelas de encaminhamento:

```
// Default constructor
public RoutingTable();

// Constructor that clones src contents
public RoutingTable(RoutingTable src);

// Test if table is initialized (it may be empty)
public boolean is_valid();

// Clear the table and stop holddown timers
public void clear();

// Add or replace a route entry to the table
public void add_route(RouteEntry re);

// Merges rt table to the local routing table
public void merge_table(RoutingTable rt);

// Delete a route entry from the table; returns true if done
public boolean delete_routeEntry(RouteEntry re);

// Returns the RouteEntry object to dest, or null if non-existing
public RouteEntry get_RouteEntry(char dest);

// Returns the next hop to reach dest, or `` if not existing
public char nextHop(char dest) {

// Returns an Entry[] with the table contents
public Entry[] get_Entry_vector();

// Returns a set with all the routes
public Collection<RouteEntry> get_routeset();

// Returns an iterator to go through the route entries
public Iterator<RouteEntry> iterator() {

// Compares rt with the routing table and returns true if equal
public boolean equal_RoutingTable(RoutingTable rt);
```

```
// Log the table contents using a log (Router) object
public void Log_routing_table(Log log);
```

A classe *Routing* define a tabela de encaminhamento do nó, guardada no objeto *tab*.

```
public RoutingTable tab; // Routing table object
```

Durante o trabalho devem ser definidos ou completados os seguintes métodos da classe *Routing*:

```
/** Envia pacotes ROUTE para todos os vizinhos */
public boolean send_local_ROUTE() {
    ... usa os métodos send_local_ROUTE_to_neighbour e
        prepare_vec_for_neighbour anteriores para enviar ... }

/** Cria a arranca relógio de envio periódico de pacotes ROUTE */
private void start_announce_timer(int duration){ ... }

/** Descodifica pacotes ROUTE e processa-os. É fornecido o código
    Que descodifica o pacote. Falta o processamento do pacote. */
public boolean process_ROUTE(char sender, DatagramPacket dp,
    String ip, DataInputStream dis) { ... }

/** Calcula a tabela de encaminhamento */
private synchronized void update_routing_table() { ... }

/** Prepara o vetor a enviar para o vizinho n */
private Entry[] prepare_vec_for_neighbour(Neighbour n) {
    ... prepara o vetor vec a enviar para um vizinho, tendo em conta os
        algoritmos de slip horizon e holddown ... }

/** Processa fim de tempo de hold down na rota re */
public boolean handle_holddown_timeout(RouteEntry re) { ... }
```

Durante o trabalho pode e deve utilizar os métodos que já são disponibilizados pelas várias classes do trabalho: para comparar tabelas de encaminhamento, comparar vetores, etc. Desta forma, **antes de iniciar o trabalho**, os alunos devem ler atentamente todo o código fornecido, de forma a poder tirar total proveito dele. Os alunos também podem modificar as classes fornecidas completas, se entenderem ser necessário.

3.4 Metas

A sequência recomendada para o desenvolvimento do programa é a seguinte:

1. Programar uma versão inicial da função *send_local_ROUTE* que envia o pacote ROUTE, na classe *Routing*;
2. Programar na classe *Routing* a função *start_announce_timer* que implementa o temporizador para envio de pacotes ROUTE e inicia a contagem de tempo;
3. Ler e compreender a classe *Neighbour*, e programar a função *process_ROUTE* da classe *Routing* para guardar os vetores de distâncias recebidos nos pacotes ROUTE dos vizinhos na lista de vizinhos;
4. Programar a função *Routing.update_routing_table* para calcular a tabela de encaminhamento aplicando o algoritmo de vetores de distâncias simples;

git: Guardar checkpoint “BasicDV”

5. Programar a variante do algoritmo com **separação de horizontes**;

git: Guardar checkpoint “SplitHorizon”

6. Programar a variante do algoritmo com **Hold down**. Para cada destino, deve ser verificado se a rota existente anteriormente deixou de ser válida, devendo-se manter a rota bloqueada durante o tempo de *hold down*, até voltar a procurar pelo destino.

git: Guardar checkpoint “Holddown”

TODOS os alunos devem tentar concluir **pelo menos a meta 5**. Na primeira semana do trabalho devem estar a trabalhar na meta 3. No fim da segunda semana devem ter terminado o passo 4. No fim da terceira semana devem estar a trabalhar no passo 6. No fim da quarta e última semana, devem tentar realizar o máximo de fases possível. Tenha em conta que é preferível fazer menos e bem (a funcionar sem erros e a conhecer o código entregue) do que fazer tudo e nada funcionar, ou pior, não conhecer o código entregue.

Postura dos Alunos

Cada grupo deve ter em consideração o seguinte:

- Não perca tempo com a estética de entrada e saída de dados
- Programe de acordo com os princípios gerais de uma boa codificação (utilização de indentação, apresentação de comentários, uso de variáveis com nomes conformes às suas funções...) e
- Proceda de modo a que o trabalho a fazer fique equitativamente distribuído pelos dois membros do grupo
- É permitida a utilização de ferramentas de inteligência artificial no desenvolvimento do código. A discussão oral final vai avaliar o nível de conhecimento dos estudantes sobre o trabalho entregue. Mais vale entregar um trabalho mais incompleto, que se conhece bem, do que um muito completo que não se conhece, o que pode levar à reprovação na unidade curricular.